Laboratoire O94.2 : validation de l'observation en action et de la programmation par composition

Dossier de tests reproductibles AAM

19 octobre 2026

Table des matières

1 Description et objectif **1**

2 Architecture testée **1**

3 Pipeline du laboratoire **1**

4 Commandes d'exécution **2**

5 Résultats observés et reçus **2**

6 Interprétation **3**

7 Conséquences théoriques, conceptuelles et opérationnelles **3**

7.1 Théoriques 3

7.2 Conceptuelles 4

7.3 Opérationnelles 4

8 Statut du modèle **4**

1 Description et objectif

Ce laboratoire corrige les défauts rapportés sur O94.1 : le message « case count 1484 exceeds configured cap 200 », l'échec d'analyse des conteneurs JSON nommés `case`, l'absence de sondage visible en streaming et l'absence d'un espace clairement identifiable pour importer les modèles locaux ou programmer par composition. Il conserve le modèle et les sources O94.1 au lieu de remplacer les fonctions historiques.

2 Architecture testée

Les composants ciblés sont : `aam_o93/blackbox.py` (lecture et capture, flux JSONL, sondes), `aam_o93/runtime.py` (gestion des sessions et fermeture sûre), `aam_o93/model_integration.py` (registres backend local/distant/adaptateur), `aam_o93/program_composer.py` (compilation sûre des compositions), `aam_o93/base_blocks.py` (briques de base), `aam_o93/ui.py` (panneaux visibles), plus le préflight et les pièces de version. La construction reste incrémentale et garde O94.1 comme parent historique.

3 Pipeline du laboratoire

1. Vérifier les hashes/CRC de l'archive HF1 O94.1 et inventorier les fichiers avant modification.

2. Tester les chargeurs sur JSON à liste racine, `case`, `cases`, dictionnaire de cas, champs standards et champs imbriqués ; tester CSV/TSV et le cap paramétré.
3. Lancer une cible de démonstration qui reste en cours d'exécution, envoyer plusieurs lignes, vérifier réponses observées, timestamps, trace JSONL et arrêt propre.
4. Générer des variantes distinctes d'une graine et vérifier que le flux les envoie véritablement à la cible ; conserver le résultat de la cible, pas une réponse attendue injectée.
5. Tester l'intégration d'un adaptateur Python local explicitement autorisé : refus avant autorisation, hash du fichier, génération, et politique d'entraînement dont le holdout est absent des données train.
6. Initialiser la UI sous Xvfb et vérifier les onglets Observer, Modèles locaux / PyTorch et Programmation par composition, ainsi que les contrôles de budget/mapping/sondes.
7. Compiler tous les fichiers Python et exécuter les lots de régression ciblés ; consigner chaque erreur puis son retest.
8. Vérifier formalisme/laboratoire, checklists, manifestes de sauvegarde, extraction des ZIP, CRC et hashes finaux.

4 Commandes d'exécution

```
python -m compileall -q aam_o93 tests/test_o942_live_observation.py
python -m pytest -q tests/test_o942_live_observation.py
python -m pytest -q tests/test_o941_blackbox.py \
  tests/test_o941_runtime_integration.py
python -m pytest -q tests/test_o941_model_integration.py
python scripts/o942_regression_batches.py \
  --batch-start 0 --batch-count 30 \
  --group-size 2 --timeout 75
PYTHONPATH=. xvfb-run -a python -m pytest -q \
  tests/test_o931_ui_smoke.py -k chat_layout
PYTHONPATH=. xvfb-run -a python -m pytest -q \
  tests/test_o931_ui_smoke.py -k trained_capsule
pdflatex -interaction=nonstopmode -halt-on-error formalisme_094_2.tex
pdflatex -interaction=nonstopmode -halt-on-error laboratoire_094_2.tex
```

La première commande de régression globale a dépassé la fenêtre du harnais et n'a pas produit de reçu agrégé. Le runner a ensuite été corrigé pour fermer le groupe de processus au timeout, puis la matrice a été exécutée en lots de deux fichiers.

5 Résultats observés et reçus

Vérification	**Résultat**	**Portée de preuve**
Lecture JSON `{"case": [...]}` sur 1484 lignes	PASS	Ingestion, alias `prompt`/`answer`/`fold`, limite UI configurable à 2000
Champs imbriqués et collection nommée	PASS	`payload.text`, collection `samples`, mappings explicites
Colonnes CSV et cap de cas	PASS	`message`/`label`, erreur de cap informative
Streaming de cible persistante	PASS	Script jouet local, deux paires réellement échangées et persistées

Sondes générées depuis graine	PASS	Mutations JSON bornées et uniques dans la ronde, sorties provenant de la cible
Capsule hors source	PASS	La trace live est ré-ingérable ; la capsule reste limitée aux entrées observées
Composition DSL	PASS	Séquence M-SEQUENCE, compétence et règle résolues, identifiant inconnu rejeté
Adaptateur PyTorch explicite simulé	PASS	Consentement obligatoire, génération, train reçoit 2 cas et exclut 1 holdout
UI premier écran	PASS individuel	Xvfb : nouveaux onglets/champs/boutons instanciés
UI sélecteur de capsule	PASS individuel	Xvfb : capsule entraînée devient sélectionnable
Compilation source Python	PASS	`compileall` sur modules modifiés et tests
Matrice de régression par lots (60 fichiers)	PASS	30/30 lots ; 349 tests et 21 sous-tests rapportés
LaTeX formalisme	PASS	Compilation pdflatex sans erreur fatale, reçu T40
LaTeX laboratoire	PASS	Compilation pdflatex sans erreur fatale, reçu T41

Les erreurs rencontrées pendant le développement ont été conservées dans les registres O94.2, avec le statut et le reçu de retest : détection initiale de `fold`, concurrence UI, assertions devenues obsolètes, chemins d'import des runners, dérive des registres, timeouts de commandes agrégées, encodage de sortie du receipt runner et syntaxe LaTeX inline. Les reçus antérieurs restent présents ; le décompte ci-dessus utilise uniquement les résultats récents qui ont passé leur contrôle pré-exécution. Les rapports détaillés sont dans `logs/O94_2/` et le registre de preuves.

6 Interprétation

L'ancienne erreur de 1484 cas était due au plafond logiciel et à une hypothèse de schéma/nom de clé : elle est résolue dans le chargeur et rendue explicite dans l'UI. Le streaming change aussi le flux d'acquisition : la cible demeure active entre les échanges, et les traces sont écrites pendant l'expérience ; le modèle n'a plus besoin d'un fichier prérempli pour le mode live. Les sondes automatiques sont des mutations de la graine et doivent être adaptées si l'objet cible a un schéma d'entrée complexe.

L'espace Modèles locaux expose le chargement Transformers de poids locaux, les endpoints compatibles Ollama/OpenAI et un contrat extensible d'adaptateur PyTorch. Un adaptateur simulé valide l'API et le respect du holdout, mais ne démontre pas la qualité d'un modèle réel. La programmation par composition devient visible et compilation/exécution passe par la DSL de briques ; il reste nécessaire de valider chaque composition sur des tâches réelles.

7 Conséquences théoriques, conceptuelles et opérationnelles

7.1 Théoriques

La capture devient un processus d'échantillonnage interactif et non seulement une importation a posteriori. La diversité d'entrée, la provenance et la séparation train/test sont des variables mesurées. Un clone est évalué à partir du domaine couvert, non inféré d'une simple création de fichier.

7.2 Conceptuelles

Les compétences effectuent des opérations, les méthodes organisent des séquences et les règles encadrent les contrats. Les modèles LLM locaux/externes fournissent des capacités spécialisées ; AAM coordonne leur emploi avec mémoire, RAG et les compositions existantes sans prétendre qu'un backend opaque est identique au runtime AAM.

7.3 Opérationnelles

Les utilisateurs voient désormais le mapping des clés et les budgets, la cible de streaming persistante, une action de sondage automatique, l'import de modèles local, et l'éditeur de composition. Le code personnalisé de modèle s'exécute seulement après autorisation explicite. Les adaptateurs GUI, DB, OCR, audio/vidéo ou matériels doivent exposer un canal d'observation approprié.

8 Statut du modèle

État : les défauts de parsing, de visibilité et d'absence d'un chemin de capture streaming sont corrigés pour les formats/protocoles documentés et les tests bornés passent. Les preuves ne montrent pas un clonage général de logiciel arbitraire ; cette capacité reste l'objectif de recherche et doit être mise à l'épreuve par des jeux d'essais indépendants et adaptateurs dédiés. Le test Windows natif, les poids externes de l'utilisateur et les périphériques réels nécessitent un environnement approprié et restent marqués non testés, sans en tirer de conclusion sur le potentiel futur d'AAM.